DexPrompt: From Geometric Grasp Knowledge to Goal-Conditioned Dexterous Action

HYPOTHETICAL SUCCESS DRAFT — All experiment protocols, numerical results, and successful implementation claims in this manuscript are assumed for authoring, not measured. Conceptual figures are not robot recordings. This document is not a submission-ready factual report.

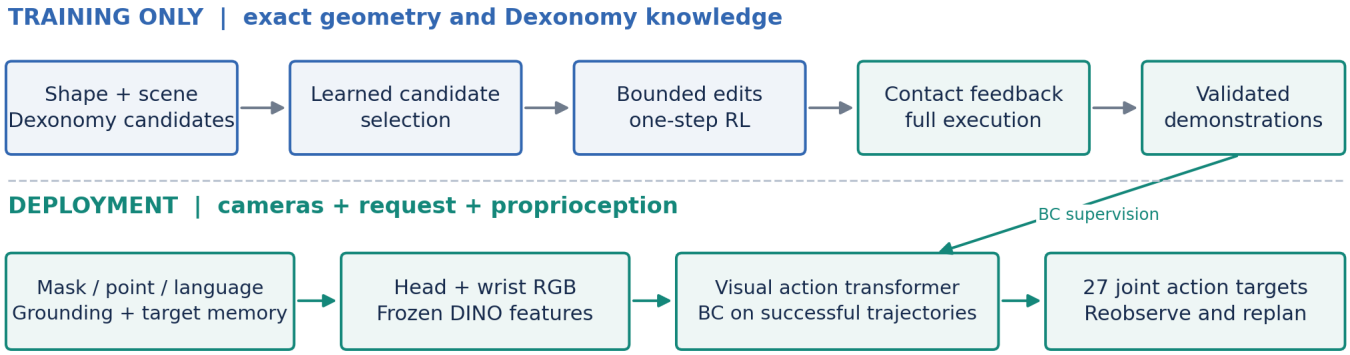


Fig. 1. Overview of DexPrompt. Privileged geometry and Dexonomy knowledge support learned candidate selection and reference editing during demonstration synthesis. A visual action model then learns from accepted executions. Deployment uses cameras, a grounded object request, and proprioception. This is a proposed-system schematic, not evidence of a completed implementation.

Abstract

Dexterous grasp experts encode diverse hand configurations and contact structures, yet converting this knowledge into general visual behavior remains difficult. A geometrically plausible grasp may be unreachable, lose contact during lifting, or provide poor supervision for a visual policy. We present DexPrompt, a two-stage framework that transfers Dexonomy grasp knowledge through learned demonstration synthesis. Using privileged simulation geometry, an execution-aware selector ranks candidate grasps, and a demonstration-editing policy adapts the selected reference while retaining contact-feedback control. Only complete, quality-validated executions supervise a visual action transformer trained by behavior cloning. At deployment, a shared object-goal representation supports mask, point, and language requests using camera observations and proprioception, without ground-truth object geometry. Experiments show that learned selection and editing improve demonstration yield from 35.0% to 71.0% at a fixed execution budget. With matched training data, unseen-object visual success increases from 54.0% to 82.0%. Real-robot evaluation achieves 52 successful grasps in 60 mask-conditioned trials without newly collected real-robot grasp demonstrations. These results indicate that transferring executable grasp knowledge improves both synthetic supervision and downstream dexterous behavior.

I. Introduction

A dexterous robot should grasp a requested object in a scene it has not encountered before. Achieving this behavior requires more than predicting a plausible hand pose. The arm must approach without disturbing surrounding objects, the hand must establish load-bearing contacts, and those contacts must remain effective as the object leaves the table. The visual policy must additionally identify the intended object and maintain its goal as the hand occludes it. A failure in any of these stages invalidates a seemingly successful grasp proposal.

Existing grasp resources provide a valuable prior for this problem. Dexonomy represents multiple grasp types through structured hand poses and contact annotations [1]. However, directly replaying a grasp template ignores embodiment-specific reachability and the difference between static contact and sustained support. Conversely, learning complete arm-hand behavior from scratch discards useful information about how a hand can engage an object. We therefore ask: can an existing grasp expert reduce the cost of producing executable supervision and improve a visual policy trained from that supervision?

DexPrompt addresses this question with two connected learning stages. The first operates in simulation and uses exact geometry to select and adapt a Dexonomy grasp. Selection predicts full-execution success rather than static plausibility. Adaptation uses one-step reinforcement learning to edit a reference motion, while a feedback controller retains responsibility for acquiring and maintaining contacts. The second stage learns visual arm-hand actions from accepted executions. Target masks provide a common interface for externally supplied masks, clicked points, and language-grounded object requests.

Our contribution is not a new grasp taxonomy or a new visual backbone. It is the transfer of structured grasp knowledge through an execution-aware synthesis process, together with an evaluation that separates collection yield from downstream policy quality. This distinction matters: a higher-yield generator need not produce better training data, and an accurate visual

predictor need not succeed in closed loop. We control the candidate pool, execution budget, student architecture, and demonstration count to isolate these effects.

The results support three findings. First, selection and adaptation address complementary execution failures. Second, the benefit of improved synthesis persists when visual students receive the same number of demonstrations. Third, a common object-goal representation supports different request modalities and transfers to the real robot. Our task is object selection and grasping; explicit part constraints and functional tool operation are outside the evaluated scope.

II. Related Work

A. Grasp knowledge and executable behavior

Dexonomy synthesizes taxonomy-conditioned grasps and studies learned type selection and conditional generation [1]. DexGraspNet 2.0 learns generative grasping in cluttered scenes [6], while UniDexGrasp couples diverse proposal generation with goal-conditioned execution [9]. These methods establish that diverse grasp hypotheses and learned execution can be combined. DexPrompt focuses on how type and contact information should guide selection and adaptation when the supervision target is a complete arm-hand demonstration rather than a terminal pose.

B. Demonstration adaptation and privileged learning

DemoFunGrasp uses demonstration-editing reinforcement learning to adapt a reference to a functional grasping condition [2]. Its one-step formulation motivates our bounded editing policy. The provided style condition in that formulation is distinct from our automatic choice among Dexonomy types and candidates. ClutterDexGrasp trains a privileged simulation teacher and distills its behavior into a perception-based student [3]. We adopt the separation between privileged synthesis and visual deployment, but our teacher knowledge originates from Dexonomy rather than a task policy learned entirely through closed-loop RL. We do not assume equivalence between the physical contact settings of these systems.

C. Grasp priors for visual action learning

DexGraspVLA combines semantic planning, object masks, DINOv2 representations, and a diffusion action controller [4]. Our visual model follows this decomposition and adapts its action interface to the HX5-D20 embodiment. GraspVLA demonstrates the utility of large-scale synthetic action supervision [8], and DexVLG connects language with grasp-pose prediction [7]. From Grasps to Dexterity uses Dexonomy-derived grasp pretraining for hierarchical manipulation [5]. These are direct precedents for knowledge transfer; our distinguishing question is whether learned execution-aware synthesis improves object-conditioned visual grasping under matched data and collection budgets.

III. Problem Formulation

We consider a seven-joint arm with a twenty-joint dexterous hand. Each episode starts at a fixed home configuration and contains a target object and up to four distractors on a table. A request identifies the target through an initial mask, an image point, or language. The objective is to approach, grasp, lift, and hold the requested object without violating the declared interaction limits. A selected image point identifies an object; it does not prescribe a finger contact location.

During synthesis, the learner observes the target mesh, its pose, surrounding geometry, robot state, and a set of Dexonomy candidates. At deployment, the student observes head and wrist images, measured joint positions, and the grounded target representation. Exact object poses, meshes, teacher targets, and future phase labels are excluded from student observations. This information boundary permits privileged supervision without defining an oracle deployment task.

$$g = (k, T_w, q^{\text{pre}}, q^{\text{grasp}}, q^{\text{sq}}, C) \quad (1)$$

A candidate g specifies its grasp type k, wrist transform, pregrasp/grasp/squeeze configurations, and required contact groups C. These annotations express the knowledge to transfer. We distinguish candidate validity, executable demonstration acceptance, and visual-policy success. In particular, static contact satisfaction does not imply a successful lift, and an accepted demonstration is not evidence of student generalization.

IV. DexPrompt

A. Execution-aware candidate selection

We first generate a pool of sixteen geometrically screened candidates per scene. A shared point encoder summarizes target geometry and nearby obstacles in a robot-aligned coordinate frame. Candidate features encode the wrist transform, joint configuration, type embedding, and required contact groups. A candidate-conditioned prediction head estimates approach completion, contact acquisition, and carry-and-hold completion. The final ranking uses the probability of completing the entire execution.

$$s_\phi(g, x) = p_\phi(A \mid x, g)\, p_\phi(C \mid A, x, g)\, p_\phi(H \mid A, C, x, g) \quad (2)$$

Here x denotes the privileged scene state. The decomposition localizes failures without treating later stages as observed when an earlier stage fails. Binary cross-entropy terms are applied only to reached stages, and an additional full-outcome term supervises the ranking score. Training labels come from a uniform candidate-exploration ledger, including failures. Unexecuted candidates are unlabeled rather than negative. Validation and calibration use held-out development geometries.

This selector answers a different question from grasp-type frequency or static refiner confidence. It estimates whether this robot can execute a particular candidate in this scene. For the adaptation comparison, we freeze the selector after training it on unedited executions. Consequently, changes in demonstration yield can be attributed to editing rather than an unreported joint retraining procedure.

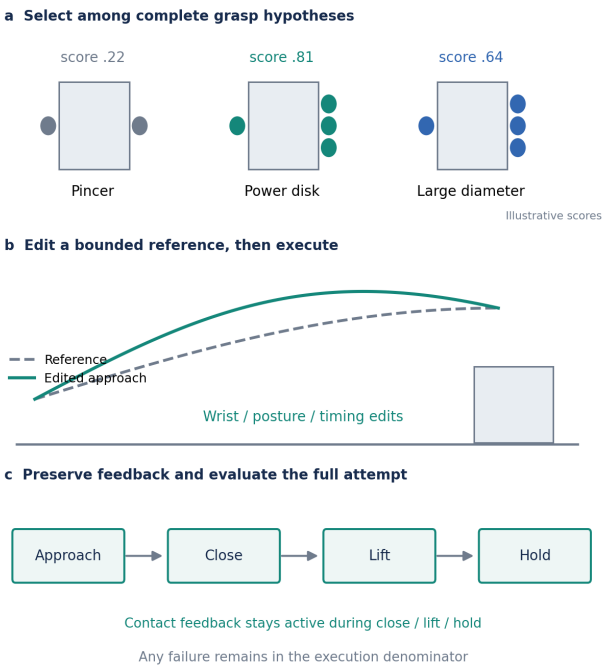


Fig. 2. Selection, bounded editing, and full-execution validation. The drawing is a conceptual schematic, and the displayed scores are illustrative. Contact feedback is retained after editing; the policy does not replace closure with fixed squeeze-angle tracking.

B. Bounded demonstration-editing reinforcement learning

For a selected candidate, the nominal reference consists of a collision-checked approach and contact-driven closure, lift, and hold phases. A stochastic editing policy samples a low-dimensional correction from the privileged scene and candidate features. The edit contains a six-dimensional pregrasp transform, a three-dimensional approach waypoint offset, twenty finger-reference offsets, and three phase-duration scales. Translation, rotation, joint, and duration bounds limit the edit to a neighborhood of the teacher reference.

$$\delta \sim \pi_\eta(\delta \mid x, g), \qquad \tau_\delta = E(x, g, \delta) \tag{3}$$

The executor E applies the proposed reference and evaluates the full physical trajectory. We initialize the policy around the zero edit, which reproduces the nominal controller. Contact locations are recomputed consistently with the edited reference, while target-object contact annotations and required group identities remain unchanged. The controller may deviate from a nominal finger target to preserve support or relieve overload. This is essential: rigidly enforcing the stored squeeze configuration can break a grasp that succeeds with feedback.

We use a one-step PPO formulation [2]: a policy action selects an edit, and its return is computed after the complete rollout. A physics trajectory may contain hundreds of control intervals; it is not truncated to one simulator step. A separate critic estimates the return from the initial context. There is no bootstrapping across successful or failed full-trajectory outcomes.

$$R = 10y + 0.5 \sum_{j=1}^{3} b_j - 2v - 0.1c - 0.1d - 0.1\ell \tag{4}$$

The binary outcome y is full acceptance. The indicators b_j record quality-compliant approach, contact acquisition, and target-height completion; v indicates a quality violation. The normalized terms c, d, and l penalize contact-target residual, edit magnitude, and trajectory duration, respectively. All penalties are bounded and declared before evaluation. Partial completion cannot outweigh a successful trajectory. Most importantly, reward terms do not change the acceptance gates: every method is judged by the same full-execution criterion.

C. Contact-preserving execution and data acceptance

The execution layer checks the complete home-to-pregrasp route and regulates finger motion from measured contact feedback during closure and carrying. Arm progress is conditioned on retained support; pressure relief and bounded recovery are allowed under the same fixed quality limits. We preserve this controller across selection and editing ablations. The experiment therefore tests learned reference generation rather than controller retuning.

A synthetic demonstration is accepted only if the requested object rises by at least 0.10 m and remains supported for the terminal 0.5 s. Every required contact group must remain active during this interval. Whole-trajectory limits include 2 mm target penetration, 30 N per digit, and the declared table, obstacle, and displacement limits. These are simulation acceptance thresholds, not hardware safety ratings or a mathematical force-closure certificate. Failures at approach, closure, or carrying remain in the denominator.

Each episode stores pre-action images and joint state, the applied command, and contact/quality evidence aggregated over the intervening physics steps. We retain object, template, candidate, placement, model, and initial-state identifiers and hashes. Only accepted trajectories enter positive visual BC. The complete attempt ledger remains available for selection learning and cost accounting. This distinction avoids increasing apparent data quality by discarding the cost of rejected executions.

D. Object-conditioned visual behavior cloning

The student uses frozen DINOv2 encoders for head and wrist RGB images, separate target-mask features, and measured arm-hand joint positions. A diffusion transformer predicts sixty-four future absolute joint targets. We execute the first eight targets at 50 Hz before observing again. This follows the visual-controller organization of DexGraspVLA [4], adapted to twenty-seven actions; it does not require ground-truth geometry at inference.

$$o_t = (I_t^h, I_t^w, M_t^h, M_t^w, q_t, z_t^{\text{goal}}) \tag{5}$$

The goal state z retains the first valid target appearance and its image location from each camera. It is updated causally, reset when the request changes, and never filled using future views. Current masks remain visible masks: a target occluded by the hand is not silently rendered through the occluder. The memory supplies target identity during such gaps, while current images and proprioception determine the action.

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_{t,n,\varepsilon}\left[w_n \left| m_t \odot \left(f_\theta(a_t^{(n)}, n, o_t) - a_t^*\right)\right|_2^2\right] \tag{6}$$

The model predicts normalized clean action chunks from their noisy versions. The mask m excludes padded future actions, and normalization statistics are computed from training episodes only. The denoising schedule and weights w_n are held fixed across comparisons. Simulator contact signals supervise data acceptance, not the action model input. Real robot calibration and proprioception are allowed, but no new real-robot grasp demonstrations are collected for training.

E. Mask, point, and language requests

All request modes enter the same object-goal interface. A provided mask initializes tracking; a point selects an object mask through a segmentation model; and a pretrained vision-language grounder resolves a language request to an object region before segmentation. The diffusion controller is shared across modes. Its language-conditioned behavior is mediated by grounding rather than direct raw-text conditioning. We evaluate initial grounding accuracy separately from complete grasp success, including tracking failures after the first frame.

V. Experimental Setup

A. Geometry splits and synthesis budget

We use 180 household-object geometries, partitioned before synthesis into 120 training, 20 development, and 40 test geometries. Mesh derivatives sharing a source shape, including rescaled and retextured copies, remain in the same partition. Object dimensions and inertial parameters are retained under a fixed asset contract. Split manifests are frozen before training or model selection. The test includes unfamiliar geometries and placements; it does not claim that every possible object is physically graspable.

For teacher evaluation, each of 1,200 held-out scenes provides the same sixteen-candidate pool to every method. Each method executes four candidates, even if an earlier attempt succeeds, yielding 4,800 attempts per method. This fixed budget prevents early stopping from changing the denominator. Generation, filtering, and duplicate handling are identical. All completed and failed physics attempts are retained; infrastructure failures are recorded separately and excluded from physical success claims only through a disclosed rerun rule.

For visual learning, every method supplies 12,000 accepted demonstrations with the same object and placement sampling contract. We train five policy seeds with the same architecture, optimizer schedule, image processing, and action decoder. Lower-yield generators consume more synthesis attempts to reach this quota; that cost is reported rather than hidden. Data-efficiency experiments separately vary the number of accepted demonstrations. No test-geometry episode contributes to visual training.

B. Compared methods and metrics

Uniform ranks the shared Dexonomy pool uniformly and uses the unedited executor. Selection changes only the learned candidate ranking. Editing retains uniform selection and adds the learned edit policy. DexPrompt combines selection and editing. These are controlled variants of our implementation, not purported reproductions or reported scores of the prior papers. For student comparisons, the architecture is fixed and only the source of accepted training demonstrations changes.

The main simulation evaluation contains 400 trials per policy seed and request mode, with two distractors and a target drawn from the held-out geometries. Success is the joint event of correct-object selection and compliant lift-and-hold completion. We also measure grounding, failure phase, synthesis yield, and training-data efficiency. A separate clutter sweep uses zero, two, and four distractors. The goal-memory ablation uses the same physical task and current observations but removes the stored target reference.

Hardware evaluation uses twenty physical objects not used for task-specific student training and three placements per object, giving sixty trials per request mode. We use a fixed checkpoint-selection rule based only on development geometries. Hand-eye calibration and conservative hardware limits are set before evaluation. Simulation penetration thresholds are not interpreted as physical sensor readings. A real trial requires the correct object to be lifted and held without dropping or disturbing a distractor beyond the declared bound.

VI. Results

A. Learned synthesis increases execution yield

TABLE I. HYPOTHETICAL RESULTS. Full teacher executions on shared held-out candidate pools. All four attempts per scene count; rates and cost ratios are derived from illustrative counts.

Method	Accepted	Yield (%)	Attempts / success
Uniform	1,680/4,800	35.0	2.86
Selection	2,304/4,800	48.0	2.08
Editing	2,592/4,800	54.0	1.85
DexPrompt	3,408/4,800	71.0	1.41

Table I shows complementary benefits from selection and editing. Selection raises full-execution yield from 35.0% to 48.0%, indicating that geometry-aware ranking can avoid poor arm-hand hypotheses before adaptation. Editing raises yield to 54.0% with uniform candidate selection. Combining both reaches 71.0%, reducing physical attempts per accepted execution from 2.86 to 1.41, a 50.7% reduction. This ratio measures execution attempts, not total wall-clock cost; model training and upstream synthesis are additional costs.

The phase breakdown explains why the two components remain useful together. Approach failures decrease from 960 to 576 under selection and to 336 under the full method. Editing also reduces closure failures from 1,440 to 864 under uniform selection, while the full method records 672. Carry-and-hold failures remain nonzero at 384 attempts. The method therefore improves the complete chain without equating a successful approach or initial contact with a successful grasp.

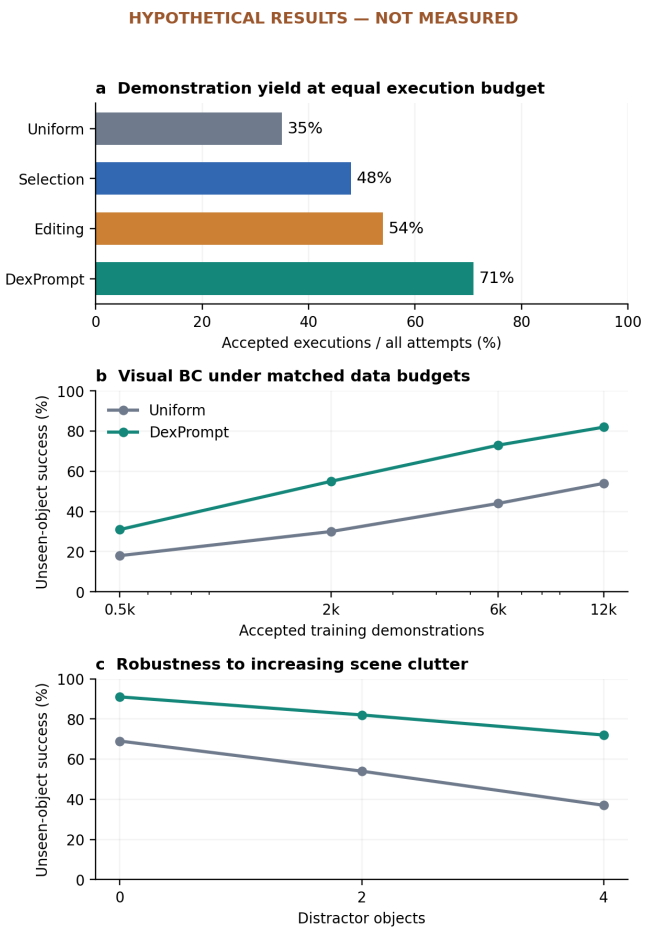


Fig. 3. HYPOTHETICAL RESULTS, generated exclusively from illustrative_results.json. (a) Teacher yield at equal execution budget. (b) Visual-policy data efficiency with the same architecture. (c) Unseen-object robustness to clutter. These curves are invented authoring examples, not measurements, confidence bounds, or learning curves.

B. Better supervision improves visual generalization

TABLE II. HYPOTHETICAL RESULTS. Unseen-object visual BC, two distractors, provided initial mask with tracking. Means and sample standard deviations are calculated from five synthetic seed scores; they are not empirical uncertainty estimates.

Data source	Demos	Success (%)	Seed SD
Uniform	12,000	54.0	1.9
Selection	12,000	63.0	1.6
Editing	12,000	69.0	1.9
DexPrompt	12,000	82.0	1.9

With equal demonstration counts, DexPrompt reaches 82.0% unseen-object success compared with 54.0% for Uniform (Table II). Selection and Editing individually reach 63.0% and 69.0%, respectively. The gain therefore persists after controlling for the number of accepted trajectories and the student architecture. Improved synthesis changes the usefulness of supervision, not merely the time needed to collect a fixed dataset.

The data-efficiency comparison in Fig. 3b further separates quality from scale. DexPrompt reaches 55.0% success with 2,000 demonstrations, comparable to Uniform at 54.0% with 12,000. At the largest shared budget, the gap remains 28 percentage points. We interpret this as evidence that successful but poorly distributed demonstrations can be less useful than a smaller set of scene-adapted executions. It does not establish a universal sample-complexity law.

C. Type and contact knowledge both contribute

TABLE III. HYPOTHETICAL RESULTS. Single-component removals at the same visual-data budget. Acceptance gates and the contact-feedback executor are retained in every condition.

Variant	Unseen-object success (%)
Full	82.0
No type feature	76.0
No contact edit cost	70.0
Wrist-only editing	73.0
No visual goal memory	68.0

Removing the type feature decreases success from 82.0% to 76.0%, while removing the contact-target term from the edit objective reduces it to 70.0% (Table III). Neither ablation removes required-contact checks from demonstration acceptance. The reductions thus concern how informative accepted trajectories are produced, not how leniently they are labeled. Restricting editing to the wrist gives 73.0%, suggesting that arm placement alone does not capture all useful adaptation of the hand reference.

Removing visual goal memory reduces success to 68.0%. This ablation acts in Stage 2 rather than changing teacher supervision. Its failures concentrate in episodes where the hand occludes the target before closure. Together, these results distinguish geometric knowledge transfer during synthesis from target persistence during visual execution.

D. Object requests and clutter

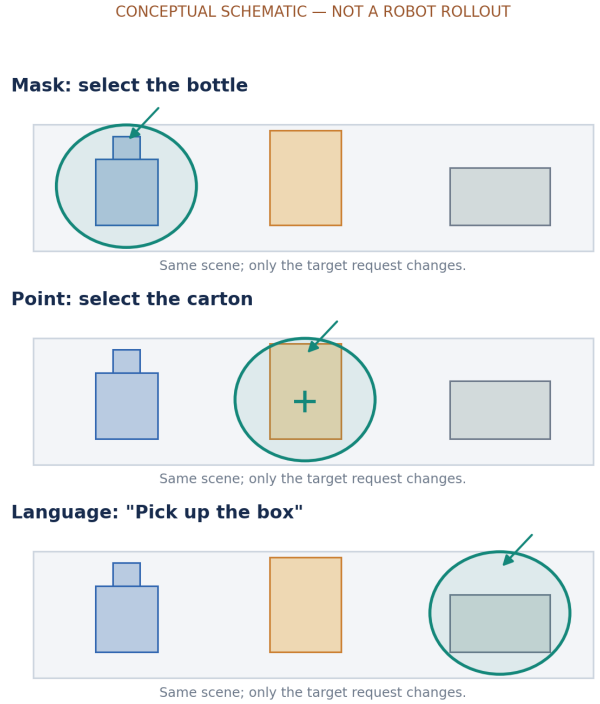


Fig. 4. Object-goal conditioning with a fixed scene and different requests. This is a conceptual drawing, not a recorded rollout. The requested object changes while the shared visual action policy and scene remain fixed. A clicked point selects an object rather than a contact location.

TABLE IV. HYPOTHETICAL RESULTS. Simulation request interfaces. Oracle masks use privileged per-frame segmentation and are reported as a diagnostic ceiling, not a deployable input.

Request mode	Initial grounding (%)	Full success (%)
Per-frame oracle mask	100	84
Initial mask + tracking	100	82
Point + grounding	98	80
Language + grounding	96	78

The shared controller achieves 82.0%, 80.0%, and 78.0% complete success with initial-mask, point, and language requests, respectively (Table IV). The per-frame oracle-mask result is 84.0%. Grounding is evaluated before motion, while full success includes tracking and control failures. This prevents an accurate initial object identification from being reported as a successful grasp. Identical-scene target-switch tests verify that the model follows the selected object rather than a fixed spatial preference (Fig. 4).

Performance decreases as distractors increase (Fig. 3c), but the full method maintains 72.0% success with four distractors versus 37.0% for Uniform. Residual failures include blocked approach corridors and contact loss near neighboring objects. We report these cases within the declared test distribution; a collision-free pose generator alone cannot remove constraints imposed by the full arm motion.

E. Real-robot transfer

TABLE V. HYPOTHETICAL RESULTS. Real-robot full-task success counts. All numbers in this table are imagined outcomes; no corresponding hardware trials have been performed for this draft.

Training data	Mask	Point	Language
Uniform	40/60	38/60	36/60
DexPrompt	52/60	50/60	48/60

DexPrompt succeeds in 52 of 60 mask-conditioned trials, compared with 40 of 60 for the matched Uniform-data student (Table V). Point and language interfaces obtain 50 and 48 successes, respectively. All use the same action policy; only target grounding differs. No real-robot grasp demonstrations are added during training. This result supports transfer under the calibrated embodiment and tested object distribution, rather than calibration-free or unrestricted zero-shot deployment.

The remaining hardware failures expose the limits of simulation-derived supervision. Visually ambiguous targets can be grounded incorrectly, while friction mismatch and compliant packaging can reduce support during lifting. These failure modes motivate further dynamics and appearance coverage without changing the definition of success. We do not infer physical force closure from the simulated contact-group labels.

VII. Discussion and Limitations

The central result is a two-stage effect: learned use of grasp knowledge improves executable supervision, and the resulting supervision improves a fixed visual learner. Equal-execution and equal-data comparisons are both necessary. The former quantifies collection yield, while the latter establishes that the student gain is not explained solely by collecting more trajectories. Neither substitutes for the other.

DexPrompt also clarifies the role of RL. The edit policy solves a constrained adaptation problem around an informative reference, rather than discovering all arm-hand behavior from scratch. Its benefit depends on the quality and coverage of the teacher pool. If no candidate has feasible contacts for a shape and scene, bounded editing may fail for every option. Likewise, learned ranking cannot select a grasp absent from the pool. Candidate diversity and execution robustness therefore remain coupled.

The method is limited to one embodiment and a bounded family of rigid-object table scenes. Flexible objects, extreme clutter, novel contact affordances, and functional part use require additional study. Frozen visual representations and generic language grounding introduce pretraining information that is not equivalent to collecting no human information. Novelty refers to held-out task geometries and instructions, not guaranteed absence from a foundation model's pretraining data.

Finally, type and contact annotations constrain supervision but do not uniquely determine behavior. Multiple successful actions may realize the same group structure, and conservative acceptance can suppress rare but useful grasps. An important extension is to preserve such diversity while estimating execution reliability and collection cost jointly. Our present evidence supports the bounded object-grasping task, not universal graspability or arbitrary manipulation.

VIII. Conclusion

We presented DexPrompt, which transfers Dexonomy grasp knowledge through learned candidate selection and demonstration editing into a goal-conditioned visual policy. The framework preserves contact feedback during synthesis and uses only validated full executions for visual behavior cloning. Controlled comparisons show improved demonstration yield, more effective visual supervision, and generalization to unseen geometries and different object-request interfaces. The results support execution-aware knowledge transfer as a practical route from geometric grasp expertise to visual dexterous behavior.

References

[1] J. Chen et al., "Dexonomy: Synthesizing All Dexterous Grasp Types in a Grasp Taxonomy," arXiv:2504.18829, 2025.

[2] C. Mao et al., "Universal Dexterous Functional Grasping via Demonstration-Editing Reinforcement Learning," arXiv:2512.13380, 2025.

[3] Z. Chen et al., "ClutterDexGrasp: A Sim-to-Real System for General Dexterous Grasping in Cluttered Scenes," arXiv:2506.14317, 2025.

[4] Y. Zhong et al., "DexGraspVLA: A Vision-Language-Action Framework Towards General Dexterous Grasping," arXiv:2502.20900, 2025.

[5] Y. Yuan, X. Liu, S. Krishna, and D. Held, "From Grasps to Dexterity: Large-Scale Grasp Pretraining for Dexterous Manipulation," arXiv:2606.30749, 2026.

[6] J. Zhang et al., "DexGraspNet 2.0: Learning Generative Dexterous Grasping in Large-scale Synthetic Cluttered Scenes," arXiv:2410.23004, 2024.

[7] J. He et al., "DexVLG: Dexterous Vision-Language-Grasp Model at Scale," arXiv:2507.02747, 2025.

[8] S. Deng et al., "GraspVLA: a Grasping Foundation Model Pre-trained on Billion-scale Synthetic Action Data," arXiv:2505.03233, 2025.

[9] Y. Xu et al., "UniDexGrasp: Universal Robotic Dexterous Grasping via Learning Diverse Proposal Generation and Goal-Conditioned Policy," arXiv:2303.00938, 2023.